Q5: Hardware Data Ingestion & Background Tasks

> ***Question:*** *How does this backend handle data coming from external hardware or background tasks? Highlight specific endpoints, background workers, or WebSockets designed to receive payloads from devices like cameras or sensors. Explain the logic used to process, parse, and store this data.*

System Overview

The backend has two entirely separate inbound data channels:

Channel	Source	Auth	Protocol
Hardware endpoints (`/api/v1/hardware/*`)	Physical IoT devices	`X-Hardware-API-Key` header	HTTP POST
WebSocket (`/ws`)	Browser clients	JWT in query param	WebSocket
APScheduler jobs	Internal cron	None (in-process)	Python function calls

Hardware devices **push** data to the API. The APScheduler **pulls** from the database on a timer. The WebSocket is used by the server to **push** results back to browser clients after hardware data is processed.

Part 1: Hardware Endpoints

Authentication Architecture

All four hardware endpoints live under one router with a **shared API key dependency**:

```
async def verify_hardware_api_key(x_hardware_api_key: Optional[str] = Header(None)):
    key = x_hardware_api_key or ""
    if not secrets.compare_digest(key, settings.HARDWARE_API_KEY):
        raise HTTPException(403, "Invalid or missing hardware API key")

router = APIRouter(dependencies=[Depends(verify_hardware_api_key)])
```

Key design choices:

- Uses `secrets.compare_digest` — constant-time string comparison to prevent timing-based key guessing
- The key is in an HTTP header (`X-Hardware-API-Key`), not a query param, so it never appears in server access logs or browser history
- The key is loaded from `.env` via `settings.HARDWARE_API_KEY` — never hardcoded
- There is no JWT, no user session, no database lookup for auth — deliberately lightweight for high-frequency device calls

---

Endpoint 1: GPS Tracking — `POST /api/v1/hardware/gps`

Rate limit: 120 requests/minute per device IP (highest of all hardware endpoints).

Pydantic schema (defined inline in `hardware.py`):

```
class GPSData(BaseModel):
    vehicle_id: int
    latitude:   float
    longitude:  float
    speed:      float   # km/h
    heading:    float = 0.0
```

Processing logic:

```
1. Validate API key
2. Pydantic coerce: vehicle_id → int, lat/lng/speed/heading → float
3. SELECT vehicle by vehicle_id → 404 if not found
4. UPDATE vehicles SET current_latitude, current_longitude (live position on map)
5. INSERT INTO gps_tracking (vehicle_id, lat, lng, speed, heading, recorded_at)
6. IF speed > 80 km/h:
     → log CRITICAL to server log (PII-redacted coordinates)
     → WebSocket broadcast_to_role("MANAGER", speed_alert payload)
7. db.commit()
8. Return HTTP 204 No Content
```

Database writes per call:

Table	Operation
`vehicles`	UPDATE `current_latitude` , `current_longitude`
`gps_tracking`	INSERT new row (append-only history log)

Over-speed alert payload sent to managers:

```
{
    "type": "speed_alert",
    "severity": "HIGH",
    "vehicle_id": <id>,
    "message": "Vehicle <id> over-speed: <speed> km/h"
}
```

The `vehicle_id` is logged but GPS coordinates are redacted from the log entry to avoid PII exposure in log files.

Endpoint 2: IoT Sensor Telemetry — `POST /api/v1/hardware/iot`

Rate limit: 60 requests/minute.

Pydantic schema:

```
class IoTSensorData(BaseModel):
```

```
vehicle_id:        int
engine_temp_c:     float
oil_pressure_psi:  float
brake_pad_mm:      float
battery_voltage:   float
```

A single POST contains **all four sensor readings at once** — not one reading per call. This reduces network round-trips from embedded hardware.

Processing logic:

```
1. Validate API key
2. Pydantic coerce all fields
3. SELECT vehicle by vehicle_id → 404 if not found
4. INSERT 4 rows into iot_sensor_readings — one per sensor type:
     ENGINE_TEMP    → data.engine_temp_c
     OIL_PRESSURE   → data.oil_pressure_psi
     BRAKE_PAD      → data.brake_pad_mm
     BATTERY_VOLTAGE → data.battery_voltage
5. Evaluate thresholds:
     engine_temp_c   > 105   → "Engine Temp CRITICAL: X° C"
     oil_pressure_psi < 25   → "Oil Pressure LOW: X PSI"
     brake_pad_mm    < 3     → "Brake Pad worn: X mm"
     battery_voltage < 11.8  → "Battery Voltage LOW: X V"
6. IF any threshold breached:
     a. SELECT first ADMIN user (system-generated request needs an owner)
     b. SELECT existing PENDING EMERGENCY request for this vehicle
     c. IF no duplicate → INSERT MaintenanceRequest:
           type=EMERGENCY, priority=1, title="Automated IoT Alert: Vehicle <plate>"
           description = all alert strings joined by " | "
     d. db.commit()
     e. WebSocket broadcast to MANAGER + ADMIN:
           { type: "iot_alert", severity: "CRITICAL", ... }
7. IF no threshold breached → db.commit() (persist the readings only)
8. Return HTTP 204 No Content
```

Threshold table:

Sensor	Field	Threshold	Condition	Alert Text
Engine Temperature	`engine_temp_c`	105°C	`>`	Engine Temp CRITICAL
Oil Pressure	`oil_pressure_psi`	25 PSI	`<`	Oil Pressure LOW
Brake Pad	`brake_pad_mm`	3 mm	`<`	Brake Pad worn
Battery Voltage	`battery_voltage`	11.8 V	`<`	Battery Voltage LOW

Duplicate prevention: Before creating a maintenance request, the endpoint checks:

```
existing_req = await db.scalar(
    select(MaintenanceRequest).where(
        MaintenanceRequest.vehicle_id == vehicle.id,
        MaintenanceRequest.status == MaintenanceStatus.PENDING,
        MaintenanceRequest.type == MaintenanceType.EMERGENCY
    )
```

```
)
if not existing_req:
    # create new request
```

This means if a vehicle keeps sending bad sensor data, only **one** pending EMERGENCY request is created — no spam.

Database writes per call (threshold breach):

Table	Operation
`iot_sensor_readings`	INSERT × 4 (one per sensor type)
`maintenance_requests`	INSERT (if no existing pending EMERGENCY)

Endpoint 3: ANPR Gate Entry/Exit — `POST /api/v1/hardware/anpr`

Rate limit: 60 requests/minute.

Pydantic schema:

```
class ANPRData(BaseModel):
    plate_number: str
    confidence:   float   # 0.0 – 1.0
    gate_id:      str
```

This is the depot entry/exit gate. An ANPR camera reads a license plate, then POSTs the result here with a confidence score.

Processing logic:

```
1. Validate API key
2. Pydantic coerce
3. Redact plate for logging: "AB***CD" (first 2 + last 2 chars)
4. IF confidence < 0.85:
     event_type = "IGNORED"
     → log info (no DB write for vehicle lookup, low-confidence reads ignored)
5. ELSE (confidence >= 0.85):
     SELECT vehicle WHERE plate_number = data.plate_number
                     AND status IN (EN_ROUTE, ASSIGNED, FREE)
     IF vehicle found:
         event_type = "GRANTED"
     ELSE:
         event_type = "DENIED"
6. INSERT INTO gate_logs (gate_id, plate_number, confidence, event, vehicle_id)
7. db.commit()
8. IF GRANTED:
     WebSocket broadcast { type: "gate_auth", event: "GATE_AUTH_GRANTED", ... }
9. IF DENIED:
     WebSocket broadcast { type: "gate_auth", event: "UNAUTHORIZED_VEHICLE", ... }
10. IF IGNORED:
     Return { "status": "ignored", "reason": "low confidence" } — no broadcast
11. Return the decision payload as HTTP 200
```

Three possible outcomes:

Outcome	Condition	DB Write	Broadcast
`GRANTED`	confidence >= 0.85 AND vehicle found with active status	`gate_logs` INSERT	MANAGER + ADMIN
`DENIED`	confidence >= 0.85 AND vehicle NOT found / wrong status	`gate_logs` INSERT	MANAGER + ADMIN
`IGNORED`	confidence < 0.85	`gate_logs` INSERT	None

The system authorises vehicles based on their **operational status** — a bus with status `MAINTENANCE` or `OUT_OF_SERVICE` will be denied even if its plate is in the database.

PII protection: Plate numbers are redacted in log entries (`AB***CD`) but stored in full in `gate_logs` for audit purposes.

Endpoint 4: In-Cabin Camera (Crowding) — `POST /api/v1/hardware/camera`

Rate limit: 60 requests/minute.

Pydantic schema:

```
class CameraData(BaseModel):
    trip_id:         int
    passenger_count: int
```

This is the most complex hardware endpoint. A YOLOv8 object-detection camera counts passengers and sends the count here. The endpoint calculates the crowding score, persists data, and may auto-dispatch a relief driver.

Processing logic:

```
1. Validate API key
2. Pydantic coerce
3. SELECT trip by trip_id → 404 if not found
4. SELECT vehicle linked to trip → 404 if not found
5. Calculate crowding score:
     IF vehicle.capacity > 0:
         crowding_score = min(passenger_count / capacity, 1.0)
     ELSE:
         crowding_score = -1.0  (invalid capacity guard)
6. Update trip in-memory (dirty mark, no SQL yet):
     trip.passenger_count = passenger_count
     trip.crowding_score  = crowding_score
     trip.is_crowded      = (crowding_score > 0.90)
7. INSERT camera_reading (trip_id, vehicle_id, passenger_count, crowding_score)
8. IF crowding_score >= 0.90 (is_crowded = True):
     CALL trigger_auto_dispatch(trip_id, db) → see breakdown below
9. IF crowding_score >= 0.70:
     INSERT crowding_event (trip_id, vehicle_id, score, count, auto_dispatch_triggered)
10. db.commit() — persists: trip UPDATE + camera_reading INSERT + crowding_event INSERT
11. IF is_crowded:
```

```
     WebSocket broadcast to MANAGER:
         { type: "crowding_alert", severity: "HIGH", crowding_score, trip_id, route_id, ... }
12. Return { "message": "Crowding updated", "score": <float> }
```

Auto-dispatch logic inside `trigger_auto_dispatch()` :

When crowding exceeds 90%, the system tries to automatically deploy a relief bus:

```
1. SELECT DRIVER_3 driver who is ACTIVE today
   WITH FOR UPDATE  ← row-level lock (prevents double-dispatch)
   → return False if no DRIVER_3 found

2. SELECT FREE vehicle
   WITH FOR UPDATE  ← row-level lock (prevents double-booking)
   → return False if no free vehicle

3. Calculate dispatch timing:
     dispatch_start = now + 5 minutes
     scheduled_end  = dispatch_start + route.estimated_time_minutes

4. INSERT extra Trip:
     trip_number = "EXT-RT{route_id}-{HHMM}-{uuid6}"
     is_extra_dispatch = True
     status = SCHEDULED

5. UPDATE driver.status = ON_TRIP
   UPDATE vehicle.status = ASSIGNED

6. db.flush()  ← sends INSERTs to DB to get new trip.id (needed for FK)

7. INSERT DriverExchange:
     reason = EMERGENCY_CROWDING
     outgoing_driver_id = original trip's driver
     incoming_driver_id = dispatched DRIVER_3
     trip_id = new_trip.id (from flush above)

8. db.commit()  ← commits the dispatch sub-transaction
9. Return True
```

Why `WITH FOR UPDATE` ?

Multiple camera frames could arrive within milliseconds of each other. Without row-level locks, two concurrent requests could both select the same DRIVER_3 and the same free vehicle, resulting in double-dispatch. The `FOR UPDATE` lock makes the second request wait until the first commits, then finds no DRIVER_3 (already dispatched) and returns `False` gracefully.

Crowding threshold summary:

Score Range	`is_crowded`	Actions Taken
< 70%	False	Update trip stats, INSERT camera_reading only
70% – 89%	False	+ INSERT crowding_event, no dispatch
>= 90%	True	+ INSERT crowding_event, auto-dispatch, WebSocket alert

Database writes per call (crowding >= 90%):

Table	Operation
`trips` (original)	UPDATE passenger_count, crowding_score, is_crowded
`camera_readings`	INSERT raw reading
`crowding_events`	INSERT event with auto_dispatch flag
`trips` (new)	INSERT extra dispatch trip
`drivers`	UPDATE status to ON_TRIP
`vehicles`	UPDATE status to ASSIGNED
`driver_exchanges`	INSERT exchange record

Part 2: Background Tasks (APScheduler)

APScheduler is started in `app/main.py` during the lifespan startup event:

```
try:
    start_scheduler()
except Exception as e:
    logger.warning(f"Scheduler failed to start (non-critical): {e}")
```

It runs **in-process** on the same event loop as FastAPI using `AsyncIOScheduler` . There is no Celery, no Redis queue, no separate worker process — everything is async coroutines inside the same application.

Three jobs are registered:

Job 1: Daily Schedule Generation — Runs at 05:30 AM

```
scheduler.add_job(
    scheduled_assignment_job,
    CronTrigger(hour=5, minute=30),
    id="daily_assignments",
    replace_existing=True
)
```

What it does:

```
1. Opens its own AsyncSession (independent of any HTTP request)
2. Calls generate_daily_schedule(db, date.today())
3. Checks for existing RotationAssignment rows for today
   → If found and regenerate=False: skip (idempotent)
   → If found and regenerate=True: delete all child records first
     (DriverExchanges → Tickets → GPSTracking → Trips → RotationAssignments)
4. Fetches all active Routes
5. Fetches all OFF_DUTY Drivers (not yet checked in)
```

```
6. Fetches all FREE Vehicles
7. For each Route × each Shift (MORNING + EVENING):
     a. Requires ≥3 drivers and ≥2 vehicles — skips if not enough
     b. Pops 3 drivers and 2 vehicles from available pools
     c. Creates 3 RotationAssignment rows (DRIVER_1, DRIVER_2, DRIVER_3)
        - DRIVER_1 and DRIVER_2 are is_active=True
        - DRIVER_3 is is_active=False (on standby)
     d. db.flush() per assignment to get IDs for Trip FK
     e. For each assignment: creates 2 Trip rows (OUTBOUND + INBOUND)
        - Scheduled times based on shift start + per-driver offset
        - DRIVER_3 trips start 1 hour later (D3_SHIFT_OFFSET_HOURS)
        - Trips that exceed shift window are skipped with a warning
8. db.commit() — persists all assignments and trips in one transaction
```

Why 05:30 AM? This is before the earliest shift start (06:00 AM), giving drivers time to check in and see their assignments.

Job 2: Rotation Processor — Runs Every 5 Minutes

```
scheduler.add_job(
    rotation_manager_job,
    'interval',
    minutes=5,
    id="rotation_processor",
    replace_existing=True,
    misfire_grace_time=300,
    max_instances=1,
)
```

`max_instances=1` prevents overlap — if a previous run takes longer than 5 minutes, the next run is skipped rather than spawning a second concurrent execution.

What it does:

```
1. Opens its own AsyncSession
2. Calls process_rotations(db)
3. SELECT all RotationAssignments active RIGHT NOW:
     shift_date = today
     AND shift_start_time <= now
     AND shift_end_time >= now
4. Groups assignments by route_id
5. For each route, extracts DRIVER_1, DRIVER_2, DRIVER_3 assignments
6. Calculates hours_since_shift_start
7. Applies ping-pong swap rules:
     1.0h ≤ elapsed < 3.0h  → if D1 is_active: swap D1 ↔ D3
     3.0h ≤ elapsed < 5.0h  → if D2 is_active: swap D2 ↔ D1
     5.0h ≤ elapsed < 7.0h  → if D3 is_active: swap D3 ↔ D2
8. _perform_swap(outgoing, incoming):
     outgoing.is_active = False
     incoming.is_active = True
     outgoing driver.status = ON_BREAK
     incoming driver.status = ACTIVE
     INSERT DriverExchange (reason=BREAK)
9. db.commit()
```

Ping-pong rotation cycle:

```
Hour 0-1:  D1 drives,  D2 on standby,  D3 on break
Hour 1-3:  D3 drives,  D1 on standby,  D2 on break
Hour 3-5:  D1 drives,  D3 on standby,  D2 on break   ← D1 rested, back in
Hour 5-7:  D2 drives,  D1 on standby,  D3 on break
```

Each swap writes one `DriverExchange` record for the audit trail.

Job 3: Midnight Reset — Runs at 00:00 AM

```
scheduler.add_job(
    midnight_reset_job,
    CronTrigger(hour=0, minute=0),
    id="midnight_reset",
    replace_existing=True
)
```

What it does:

```
1. Opens its own AsyncSession
2. SELECT all Driver rows
3. For every driver:
     total_trips_today       = 0
     break_time_remaining    = settings.BREAK_TIME_PER_SHIFT  (default 60 min)
     total_break_time_today  = 0.0
     trips_since_last_break  = 0
     current_break_number    = 0
4. db.commit()
```

This resets the daily counters so break eligibility and trip counts are fresh for the next shift. Drivers who are still on an overnight shift retain their status — only the counters are reset.

Part 3: WebSocket as Outbound Channel

The WebSocket (`/ws`) does not receive hardware data — it is used exclusively by the backend to **push events to browser clients** after hardware data has been processed.

How hardware data becomes a WebSocket event:

```
Hardware device POSTs data
         ↓
Hardware endpoint processes + commits to DB
         ↓
Endpoint calls: await manager.broadcast_to_role("MANAGER", payload)
         ↓
ConnectionManager iterates active_connections["MANAGER"]
         ↓
websocket.send_json(payload) to each connected manager browser tab
```

All hardware-triggered WebSocket events:

Source Endpoint	Event Type	Recipients	Trigger Condition
`/hardware/gps`	`speed_alert`	MANAGER	Speed > 80 km/h
`/hardware/iot`	`iot_alert`	MANAGER + ADMIN	Any sensor threshold breached
`/hardware/anpr`	`gate_auth` (GRANTED)	MANAGER + ADMIN	Confidence >= 0.85, vehicle found
`/hardware/anpr`	`gate_auth` (DENIED)	MANAGER + ADMIN	Confidence >= 0.85, vehicle NOT found
`/hardware/camera`	`crowding_alert`	MANAGER	Crowding score >= 90%

Connection management in `ConnectionManager` :

```
active_connections: Dict[str, Set[WebSocket]] = {
    "ADMIN":   set(),
    "MANAGER": set(),
    "DRIVER":  set(),
}
```

The WebSocket endpoint (`/ws`) authenticates via JWT and adds the connection to the correct role bucket. When `broadcast_to_role` is called, it iterates the set and removes any sockets that throw exceptions (stale connections):

```
async def broadcast_to_role(self, role: str, message: dict):
    disconnected = set()
    for connection in tuple(self.active_connections[role]):
        try:
            await connection.send_json(message)
        except Exception:
            disconnected.add(connection)
    for conn in disconnected:
        self.active_connections[role].discard(conn)
```

Redis Pub/Sub relay (for multi-process scaling):

```
await self.pubsub.subscribe("broadcast", "alerts")
```

The `ConnectionManager` also subscribes to two Redis channels on startup. Any message published to these channels is forwarded to the appropriate WebSocket connections. This means other parts of the application (or future worker processes) can publish to Redis to reach browser clients without directly calling `broadcast_to_role` .

The Redis listener runs as a background `asyncio.Task` with a self-healing supervisor — if Redis disconnects, it waits 5 seconds and reconnects automatically.

Summary: Data Flow by Source

```
PHYSICAL HARDWARE
```

```
   GPS Unit          → POST /hardware/gps    → UPDATE vehicles + INSERT gps_tracking
   IoT Sensors       → POST /hardware/iot    → INSERT iot_sensor_readings (×4) + optional MaintenanceRequest
   ANPR Camera       → POST /hardware/anpr   → INSERT gate_logs
   In-Cabin Camera   → POST /hardware/camera → UPDATE trips + INSERT camera_readings + optional auto-dispatc

INTERNAL SCHEDULER (APScheduler)
   CronJob 05:30     → generate_daily_schedule()  → INSERT rotation_assignments + trips
   Interval 5 min    → process_rotations()         → UPDATE assignments + INSERT driver_exchanges
   CronJob 00:00     → midnight_reset_job()        → UPDATE all drivers (reset daily counters)

REAL-TIME OUTPUT (WebSocket)
   After any hardware event → broadcast_to_role() → JSON push to Manager/Admin browser clients
   Via Redis Pub/Sub         → any process can publish → routed to correct role bucket
```

Generated: 2026-04-16